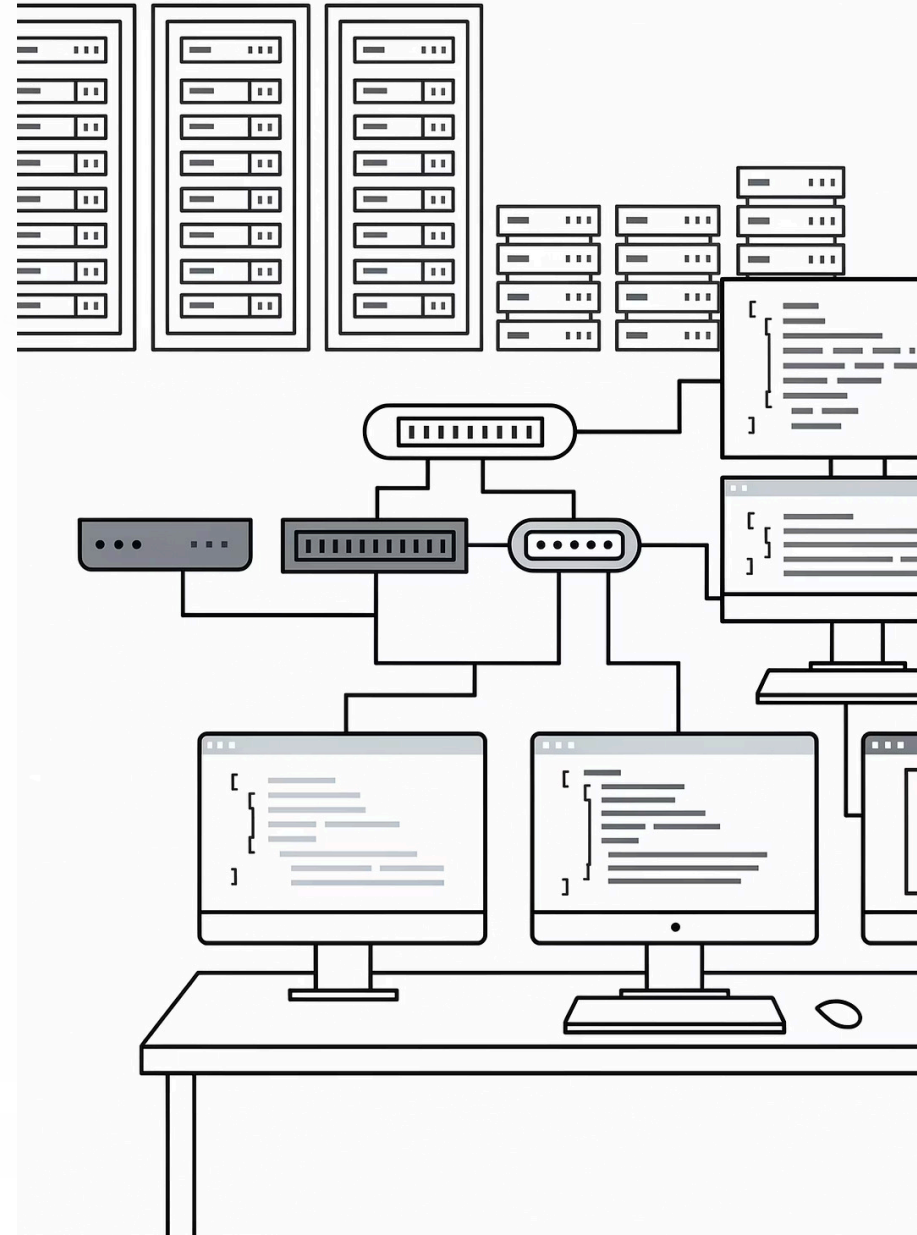


Spring Data

Développer des applications d'entreprise

Persistence des données, transactions, optimisation des performances et manipulation avancée des données.



Prérequis du module

Ces compétences constituent les fondations sur lesquelles repose ce module.

Java EE

Servlets, JSP, requêtes HTTP,
architecture MVC.

Spring Boot

Auto-configuration, démarrage
rapide, IoC container, annotations
@Component, @Autowired.

AOP

Aspects, pointcuts, advices — logging
et sécurité transversale.

Plan du module

01

Persistence avec Spring Data JPA

Configuration, mapping ORM, repositories, relations, héritage.

03

Gestion des transactions

Propagation, isolation, rollback, erreurs, transactions distribuées.

02

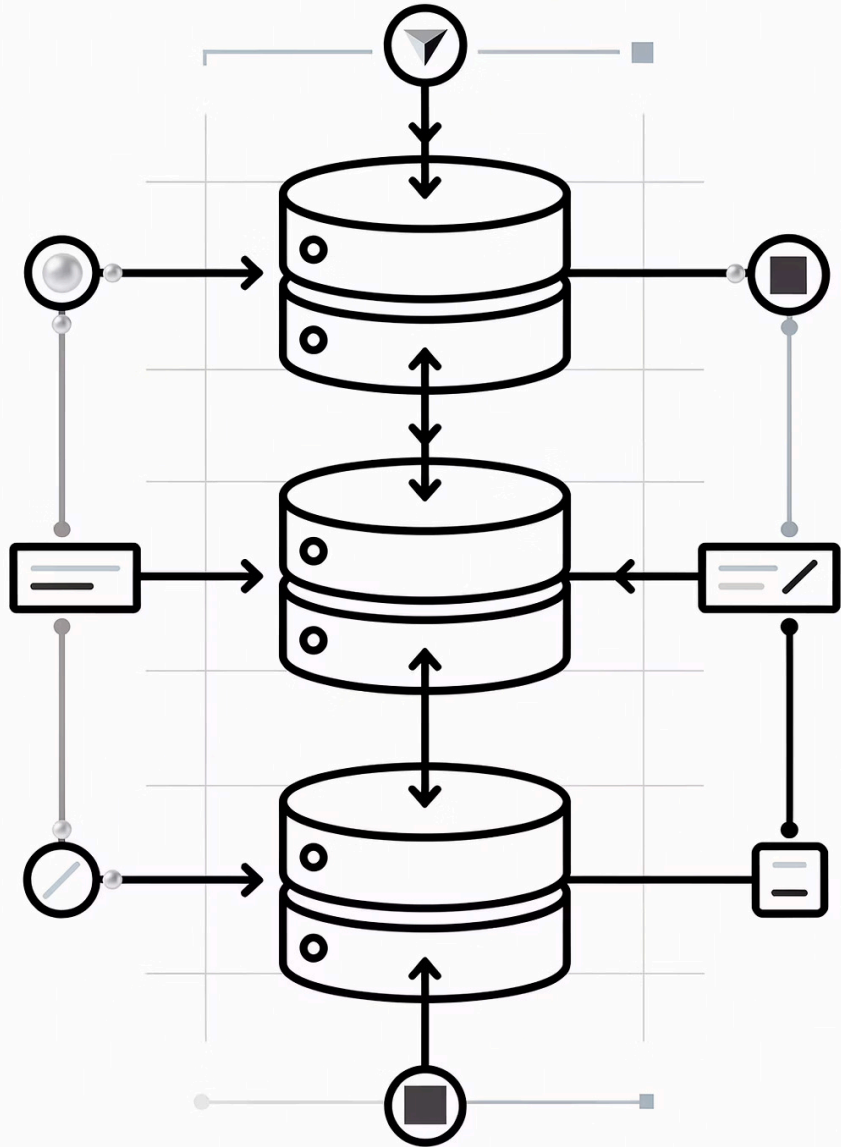
Requêtes et manipulation des données

Query methods, JPQL, SQL natif, projections, pagination, critères dynamiques.

04

Performance et optimisation

Lazy/eager loading, caching, monitoring, bonnes pratiques.

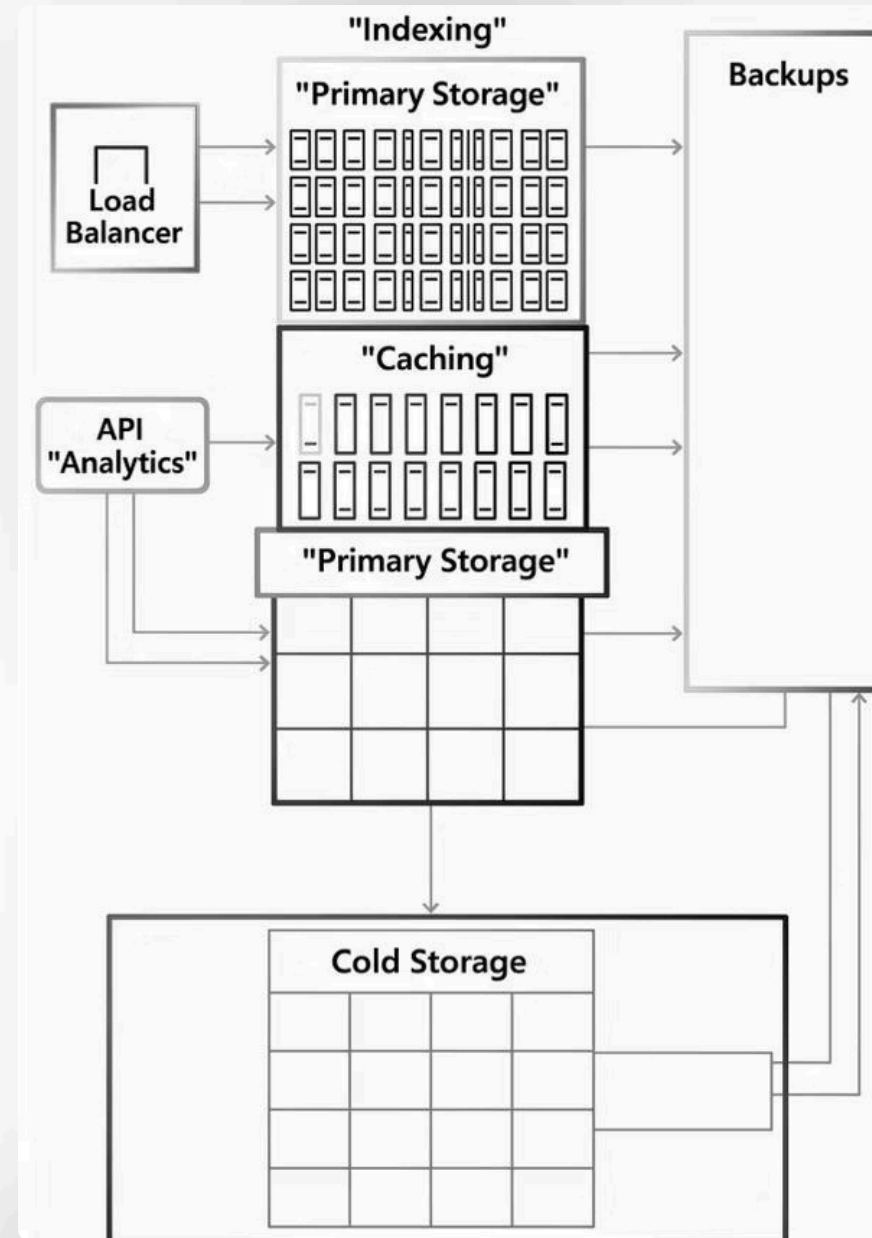


PARTIE 1

Persistence avec Spring Data JPA

Architecture de persistance

Spring Data JPA simplifie l'interaction avec les bases de données relationnelles en s'appuyant sur JPA et Hibernate — réduisant le code boilerplate et offrant un modèle de programmation cohérent.



Qu'est-ce que JPA ?

Sans Spring Data JPA

- EntityManager manuel
- Transactions gérées à la main
- JPQL écrit entièrement par le développeur
- Beaucoup de code répétitif

Avec Spring Data JPA

- Repositories auto-générés
- Transactions via annotations
- Query methods dérivées automatiquement
- Code minimal, productivité maximale

L'implémentation JPA la plus répandue dans l'écosystème Spring est **Hibernate**, qui traduit les opérations JPA en requêtes SQL spécifiques à chaque base de données.

Configuration de la persistance

Le fichier `application.properties` centralise toute la configuration JPA : URL de connexion, identifiants, dialecte SQL et stratégie de gestion du schéma.

```
spring.datasource.url=jdbc:postgresql://localhost:5432/monapp
spring.datasource.username=postgres
spring.datasource.password=secret
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
spring.jpa.database-platform=org.hibernate.dialect.PostgreSQLDialect
```

 La valeur `ddl-auto=update` est pratique en développement mais ne doit jamais être utilisée en production. Préférez `validate` ou gérez le schéma avec Flyway/Liquibase.

Mapping Objet-Relationnel (ORM)

Les annotations JPA établissent la correspondance entre classes Java et tables de base de données.

```
@Entity
@Table(name = "produits")
public class Produit {
    @Id
    @GeneratedValue(strategy =
        GenerationType.IDENTITY)
    private Long id;

    @Column(name = "nom_produit",
        nullable = false, length = 200)
    private String nom;

    @Column(precision = 10, scale = 2)
    private BigDecimal prix;
}
```

@Entity / @Table

Déclarent la classe persistante et la table cible.

@Id / @GeneratedValue

Clé primaire. Stratégie SEQUENCE recommandée avec PostgreSQL.

@Column

Nom, nullabilité, longueur, unicité.

@Transient

Exclut un champ de la persistance.

Annotations ORM avancées



@Temporal / @Enumerated

Précise le type de date SQL ou si un enum est stocké en valeur ordinaire ou chaîne.



@Embeddable / @Embedded

Incorporent un objet valeur dans une entité — idéal pour regrouper des champs liés (adresse, coordonnées).



@GeneratedValue SEQUENCE

Stratégie recommandée pour les performances avec PostgreSQL et les opérations en batch.

Repositories Spring Data

Les repositories éliminent le besoin d'implémenter manuellement les opérations CRUD. Il suffit de déclarer une interface.

```
@Repository
public interface ProduitRepository
    extends JpaRepository<Produit, Long> {
    // save(), findById(), findAll(),
    // delete(), count(), existsById()...
}
```

1

CrudRepository

Opérations CRUD simples. Point de départ minimal.

2

PagingAndSortingRepository

Ajoute pagination (Pageable) et tri (Sort).

3

JpaRepository

Interface recommandée. Ajoute flush(), saveAndFlush(), deleteInBatch().

Query Methods et dérivation automatique

Spring Data JPA génère la requête JPQL à partir du nom de la méthode — sans annotation supplémentaire.

```
// SELECT * FROM produits WHERE nom = ?
```

```
List<Produit> findByNom(String nom);
```

```
// SELECT * FROM produits WHERE prix < ? AND disponible = true
```

```
List<Produit> findByPrixLessThanAndDisponibleTrue(BigDecimal prix);
```

```
// SELECT * FROM produits WHERE nom LIKE %?%
```

```
List<Produit> findByNomContainingIgnoreCase(String motCle);
```

```
// SELECT COUNT(*) FROM produits WHERE categorie = ?
```

```
long countByCategorie(String categorie);
```

Mots-clés de dérivation

Mot-clé	Exemple	JPQL
And / Or	findByNomAndPrix	WHERE nom=? AND prix=?
Between	findByPrixBetween	WHERE prix BETWEEN ? AND ?
LessThan / GreaterThan	findByPrixLessThan	WHERE prix < ?
Like / Containing	findByNomContaining	WHERE nom LIKE %?%
IsNull / IsNotNull	findByDescriptionIsNull	WHERE description IS NULL
In / NotIn	findByCategorieIn	WHERE categorie IN (?)
True / False	findByDisponibleTrue	WHERE disponible = true
Top / First	findTop3ByCategorie	LIMIT 3

Relations entre entités



@OneToOne

Un Utilisateur possède un seul ProfilDetail. Utiliser @JoinColumn pour la clé étrangère.

@OneToMany / @ManyToOne

Une commande contient plusieurs lignes. Relation bidirectionnelle gérée avec mappedBy.

@ManyToMany

Produits et catégories. Nécessite une table de jointure avec @JoinTable.

Relations bidirectionnelles – exemple

@Entity

```
public class Commande {  
    @OneToMany(mappedBy = "commande",  
        cascade = CascadeType.ALL,  
        orphanRemoval = true)  
    private List<LigneCommande> lignes = new ArrayList<>();  
}
```

@Entity

```
public class LigneCommande {  
    @ManyToOne(fetch = FetchType.LAZY)  
    @JoinColumn(name = "commande_id")  
    private Commande commande;  
    private int quantite;  
    private BigDecimal prixUnitaire;  
}
```

 **Attention aux cascades :** CascadeType.ALL inclut REMOVE — toutes les lignes seront supprimées si la commande l'est.
orphanRemoval = true supprime les lignes non référencées.

Héritage et stratégies de mapping

@Entity

@Inheritance(strategy =
InheritanceType.SINGLE_TABLE)

@DiscriminatorColumn(
name = "type_paiement")
public abstract class Paiement { ... }

@Entity

@DiscriminatorValue("CARTE")
public class PaiementCarte
extends Paiement { ... }

@Entity

@DiscriminatorValue("VIREMENT")
public class PaiementVirement
extends Paiement { ... }

SINGLE_TABLE

Une seule table avec
colonne discriminante.
Requêtes simples, pas de
jointures.

TABLE_PER_CLASS

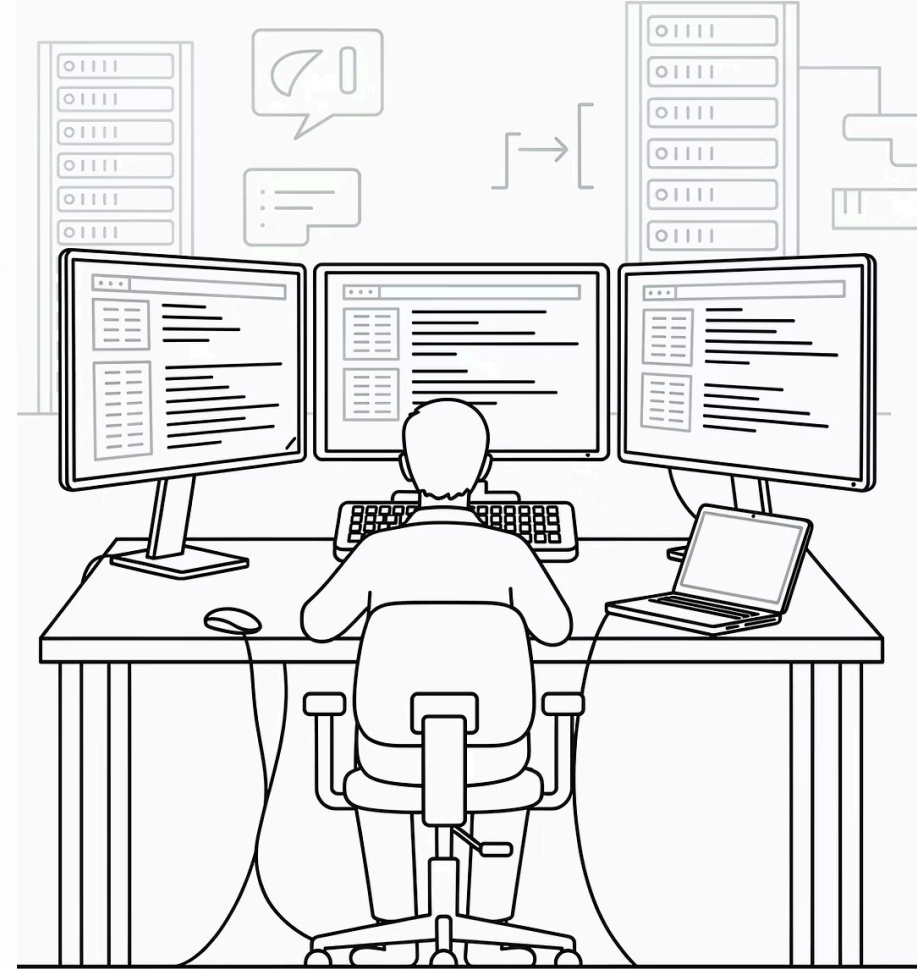
Chaque sous-classe a sa
propre table. Requêtes
polymorphiques
coûteuses (UNION).

JOINED

Schéma normalisé. Entités reconstruites par jointure à
chaque lecture.

PARTIE 2

Requêtes et manipulation des données



Query Methods avancées avec @Query

La dérivation couvre les cas simples à modérément complexes. Pour les requêtes élaborées, `@Query` permet de fournir directement du JPQL ou du SQL natif.

```
// Dérivation avec tri et limitation  
List<Commande> findTop10ByStatutOrderByDateCreationDesc(String statut);
```

```
// JOIN FETCH avec paramètres nommés  
@Query("SELECT c FROM Commande c JOIN FETCH c.lignes " +  
"WHERE c.client.id = :clientId AND c.dateCreation > :depuis")  
List<Commande> findCommandesAvecLignes(  
    @Param("clientId") Long clientId, @Param("depuis") LocalDate depuis);
```

```
// Requête de modification  
@Modifying  
@Query("UPDATE Commande c SET c.statut = :statut WHERE c.id = :id")  
int updateStatut(@Param("id") Long id, @Param("statut") String statut);
```

Requêtes JPQL

JPQL opère sur les entités Java plutôt que sur les tables SQL — les requêtes sont portables et indépendantes du SGBD.

// Requête avec plage de prix

```
@Query("SELECT p FROM Produit p WHERE p.prix BETWEEN :min AND :max")
List<Produit> findByPrixRange(@Param("min") BigDecimal min,
                             @Param("max") BigDecimal max);
```

// JOIN FETCH pour éviter le problème N+1

```
@Query("SELECT c FROM Categorie c LEFT JOIN FETCH c.produits " +
        "WHERE c.actif = true ORDER BY c.nom")
List<Categorie> findCategoriesActives();
```

// Agrégation

```
@Query("SELECT c.statut, COUNT(c) FROM Commande c GROUP BY c.statut")
List<Object[]> countByStatut();
```

❗ JOIN FETCH vs JOIN : Un simple JOIN filtre les résultats mais ne charge pas les données associées (lazy). JOIN FETCH charge tout en une seule requête SQL.

Requêtes SQL Natives

Certaines situations nécessitent du SQL natif : fonctionnalités SGBD spécifiques (JSON PostgreSQL, Full-Text Search), optimisations très ciblées, ou vues complexes existantes.

```
// SQL natif simple
@Query(value = "SELECT * FROM produits WHERE LOWER(nom) LIKE %:motCle%",
nativeQuery = true)
List<Produit> searchByNomNatif(@Param("motCle") String motCle);
```

```
// SQL natif avec pagination (countQuery obligatoire)
@Query(value = "SELECT * FROM produits p JOIN categories c " +
"ON p.categorie_id = c.id WHERE c.nom = :categorie",
countQuery = "SELECT COUNT(*) FROM produits p JOIN categories c " +
"ON p.categorie_id = c.id WHERE c.nom = :categorie",
nativeQuery = true)
Page<Produit> findByCategorieNom(@Param("categorie") String cat,
Pageable pageable);
```

Projections et DTO

Charger une entité complète quand on n'a besoin que de quelques champs est une source fréquente de gaspillage. Les projections réduisent la consommation mémoire et le volume de données transféré.

```
// Interface-based projection
public interface ProduitResume {
    String getNom();
    BigDecimal getPrix();
}

List<ProduitResume> findByCategorie(String categorie);

// DTO class-based avec @Query
@Query("SELECT new com.app.dto.ProduitDTO(p.id, p.nom, p.prix) " +
    "FROM Produit p WHERE p.disponible = true")
List<ProduitDTO> findProduitsDisponiblesDTO();
```

- ❏ Les projections par interface supportent les **Open Projections** avec @Value pour des champs calculés côté Spring — attention : charge l'entité complète en mémoire.

Pagination et tri

```
// Repository
Page<Produit> findByCategorie(
    String categorie, Pageable pageable);

// Service
Pageable pageable = PageRequest.of(
    0, 20, Sort.by("prix").descending());
Page<Produit> page =
    produitRepository.findByCategorie(
        "Electronique", pageable);

long total = page.getTotalElements();
int totalPages = page.getTotalPages();
List<Produit> produits = page.getContent();
boolean hasNext = page.hasNext();
```

Page<T>

Données + métadonnées complètes (total, pages).
Exécute un COUNT supplémentaire.

Slice<T>

Sait seulement s'il y a une page suivante. Plus léger — idéal pour le défilement infini.

List<T>

Simple liste sans métadonnées. Limitation/offset sans informations de total.

Spécifications et critères dynamiques

Quand les critères de recherche sont déterminés au runtime, les **Specifications** permettent de construire des filtres multicritères composables via `and()`, `or()`, `not()`.

```
public interface ProduitRepository extends JpaRepository<Produit, Long>,
    JpaSpecificationExecutor<Produit> {}

public class ProduitSpecifications {
    public static Specification<Produit> avecCategorie(String cat) {
        return (root, query, cb) ->
            cat == null ? null : cb.equal(root.get("categorie"), cat);
    }
    public static Specification<Produit> prixEntre(BigDecimal min, BigDecimal max) {
        return (root, query, cb) -> cb.between(root.get("prix"), min, max);
    }
}

// Utilisation dynamique
Specification<Produit> spec = Specification
    .where(avecCategorie(filtre.getCategorie()))
    .and(prixEntre(filtre.getPrixMin(), filtre.getPrixMax()));
Page<Produit> resultats = produitRepository.findAll(spec, pageable);
```

Gestion des transactions



Les propriétés ACID



Atomicité

Tout réussit ou tout est annulé. En cas d'échec, rollback complet.



Cohérence

La base passe d'un état valide à un autre. Les contraintes d'intégrité sont toujours respectées.



Isolation

Les transactions concurrentes n'interfèrent pas entre elles.



Durabilité

Une fois committée, la transaction survit aux pannes et redémarrages.

Configuration avec @Transactional

Spring génère un proxy AOP qui intercepte les appels de méthode pour gérer le cycle de vie de la transaction de façon transparente.

```
@Service
@Transactional
public class CommandeService {

    @Transactional(readOnly = true) // Optimisation lectures
    public List<Commande> findAll() {
        return commandeRepository.findAll();
    }

    @Transactional
    public Commande creerCommande(CommandeDTO dto) {
        Commande commande = mapper.toEntity(dto);
        stockService.reserverStock(dto.getLignes());
        return commandeRepository.save(commande);
    }

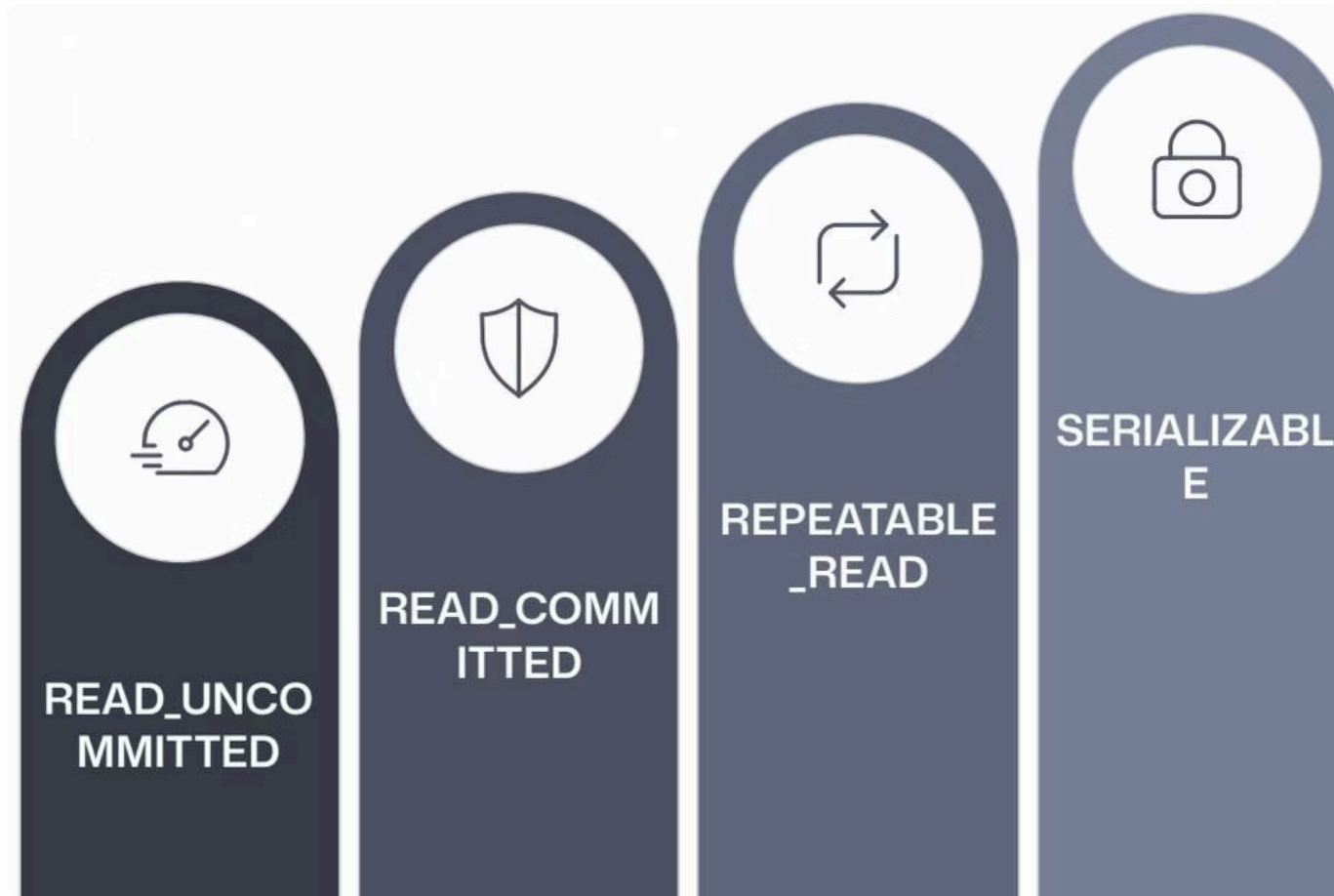
    @Transactional(rollbackFor = Exception.class)
    public void traiterPaieement(Long id, PaiementDTO p) { ... }
}
```

Propagation des transactions

Mode	Comportement	Usage typique
REQUIRED (défaut)	Utilise la transaction existante ou en crée une	Service standard
REQUIRES_NEW	Suspend la courante, crée une indépendante	Logging, audit
NESTED	Crée un savepoint (rollback partiel possible)	Opérations partielles
SUPPORTS	Utilise si disponible, sinon sans transaction	Lecture optionnelle
MANDATORY	Requiert une transaction existante	Méthodes internes
NEVER	Lève une exception si transaction active	Opérations sensibles

 **Piège REQUIRES_NEW :** Si la transaction parente effectue un rollback, les modifications de la transaction enfant (déjà committée) ne seront pas annulées.

Niveaux d'isolation



Un niveau d'isolation élevé offre plus de cohérence mais réduit la concurrence. Les trois anomalies classiques :

Le niveau par défaut Spring est **DEFAULT**, qui délègue au SGBD (généralement **READ_COMMITTED**).

Dirty Read

Lire des données non encore committées.

Non-Repeatable Read

Une même lecture donne des résultats différents dans la même transaction.

Phantom Read

De nouvelles lignes apparaissent lors d'une re-lecture.

Gestion des erreurs et rollback

Par défaut, Spring rollback uniquement sur les `RuntimeException`. Les checked exceptions ne déclenchent *pas* de rollback automatiquement.

```
@Transactional(
    rollbackFor = {Exception.class, BusinessException.class},
    noRollbackFor = {ValidationWarningException.class}
)
public void processOrder(OrderDTO dto) throws BusinessException {
    try {
        Order order = createOrder(dto);
        paymentService.charge(order);
    } catch (PaymentException e) {
        // Rollback sans relancer
        TransactionAspectSupport.currentTransactionStatus()
            .setRollbackOnly();
        auditService.logFailure(dto, e); // Transaction séparée
        throw new BusinessException("Paiement échoué", e);
    }
}
```

Transactions distribuées

1

JTA (Java Transaction API)

Protocole standard avec gestionnaire global (Atomikos, Narayana).
Implémente le **2PC (Two-Phase Commit)**. Garantit l'ACID total — coût de performance élevé.

2

Pattern Saga

Alternative moderne au 2PC. Chaque étape locale a une opération de compensation. Variantes : **Choreography** et **Orchestration**.
Meilleure scalabilité, cohérence éventuelle.

3

Outbox Pattern

Garantit la cohérence entre transaction en base et envoi de messages. Les événements sont persistés dans une table "outbox" puis relayés de façon asynchrone.

 Dans la grande majorité des applications Spring monolithiques, `@Transactional` standard couvre la quasi-totalité des besoins.

Pièges courants avec @Transactional

→ Auto-invocation

Appeler une méthode @Transactional depuis la même classe contourne le proxy AOP — la transaction ne sera pas active.

→ LazyInitializationException

Accéder à une collection LAZY hors transaction lève cette exception. Solutions : JOIN FETCH, projection, ou ouvrir une transaction.

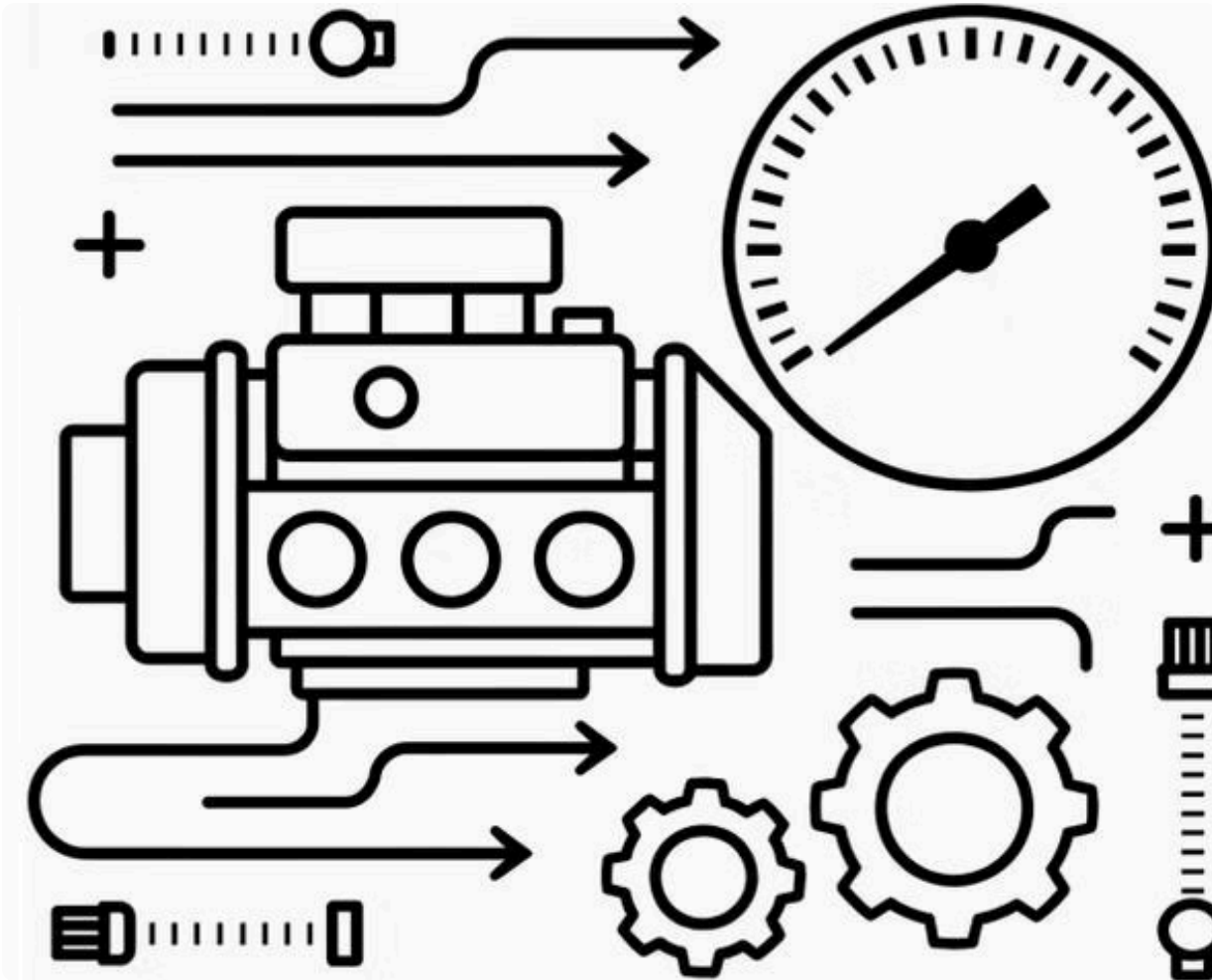
→ Méthodes privées

Spring AOP ne peut pas intercepter les méthodes privées. L'annotation est silencieusement ignorée.

→ Rollback sur checked exceptions

Toujours spécifier `rollbackFor = Exception.class` si vous voulez un rollback systématique.

Performance et optimisation



Le problème N+1

L'une des causes les plus fréquentes de dégradation des performances avec JPA. Pour une liste de 100 commandes, cela représente **101 requêtes SQL** au lieu d'une seule.

```
// Code innocent en apparence... DÉSASTREUX en production !  
List<Commande> commandes = commandeRepository.findAll(); // 1 requête  
for (Commande c : commandes) {  
    System.out.println(c.getClient().getNom()); // N requêtes !  
}  
// Total : N+1 requêtes SQL
```

 **Signe d'alarme** : si le nombre de requêtes SQL est proportionnel au nombre d'éléments retournés, vous avez très probablement un problème N+1. Un test avec 1000 éléments peut transformer une API de 50ms en 5 secondes.

Lazy vs Eager Fetching

Lazy Loading ✓ Recommandé

L'association n'est chargée que si on y accède. Réduit les données inutiles.

- @OneToMany : LAZY par défaut ✓
- @ManyToMany : LAZY par défaut ✓
- Utiliser JOIN FETCH si besoin

Eager Fetching ⚠ À éviter

L'association est toujours chargée, même si inutile. Peut multiplier les données retournées.

- @ManyToOne : EAGER par défaut ⚠
- @OneToOne : EAGER par défaut ⚠
- Forcer LAZY si non systématiquement nécessaire

Solutions au problème N+1

JOIN FETCH

Charge l'entité et son association en une seule requête SQL. Utiliser DISTINCT avec @OneToMany pour éviter les doublons.

```
@Query("SELECT DISTINCT c  
FROM Commande c  
JOIN FETCH c.lignes  
WHERE c.statut = :s")
```

@EntityGraph

Alternative déclarative à JOIN FETCH. Spécifie les associations à charger sans modifier la requête JPQL.

```
@EntityGraph(attributePaths =  
    {"lignes", "client"})  
List<Commande> findByStatut(  
    String statut);
```

Projections / DTO

La solution la plus performante pour les lectures. Sélectionner uniquement les champs nécessaires évite totalement le chargement des associations inutiles.

Caching avec Spring

```
@Configuration
@EnableCaching
public class CacheConfig {
    @Bean
    public CacheManager cacheManager() {
        CaffeineCacheManager manager = new CaffeineCacheManager();
        manager.setCaffeine(Caffeine.newBuilder()
            .maximumSize(500)
            .expireAfterWrite(10, TimeUnit.MINUTES));
        return manager;
    }
}

@Service
public class CategorieService {
    @Cacheable("categories")    // Mise en cache
    public List<Categorie> findAll() { ... }

    @CacheEvict(value = "categories", allEntries = true) // Invalidation
    public Categorie save(Categorie categorie) { ... }

    @CachePut(value = "categories", key = "#result.id") // Mise à jour
    public Categorie update(Categorie categorie) { ... }
}
```

Caffeine est recommandé pour les caches locaux. Redis est préféré pour les architectures distribuées ou multi-instances.

Cache L1 et L2 d'Hibernate

1

Cache L1 – Session Cache

Actif par défaut. Chaque transaction maintient un cache en mémoire. La même entité chargée deux fois dans la même transaction ne génère qu'une seule requête SQL.

2

Cache L2 – Second Level Cache

Cache partagé entre sessions, à activer explicitement (EhCache, Caffeine, Hazelcast). Très efficace pour les entités peu modifiées. Configurable par entité avec `@Cache`.

3

Query Cache

Cache les résultats des requêtes JPQL. Activer avec `hibernate.cache.use_query_cache=true`. Limité aux entités immuables ou rarement modifiées.

Optimisation des requêtes JPA



Opérations en batch

Activer `hibernate.jdbc.batch_size=50` pour regrouper les insertions. Combiner avec `saveAll()` et `SEQUENCE`.



Sélection de colonnes

Éviter le `SELECT *` implicite. Utiliser projections et DTO, surtout pour les tables avec colonnes LOB.



Indexation

Ajouter des index sur les colonnes utilisées dans `WHERE`, `JOIN`, `ORDER BY` via `@Table(indexes = @Index(...))`.



Mises à jour bulk

Préférer `@Modifying @Query("UPDATE...")` aux opérations entité par entité — exponentiellement plus rapide.

Monitoring et Profiling

Spring Boot Actuator

Métriques JVM, BDD, pool de connexions et caches via HTTP.
Intégrable avec Prometheus et Grafana.

Hibernate Statistics

Statistiques détaillées sur les requêtes SQL et hits/misses de cache. Activer avec `hibernate.generate_statistics=true`.

P6Spy / Datasource Proxy

Interceptent toutes les requêtes SQL au niveau JDBC. Loggent avec les paramètres réels et mesurent les temps d'exécution.

Slow Query Log

Configurable au niveau SGBD (PostgreSQL : `log_min_duration_statement`). Enregistre les requêtes dépassant un seuil.

Bonnes pratiques de performance



Toujours FetchType.LAZY par défaut

Forcer LAZY sur toutes les associations, y compris @ManyToOne. Charger EAGER uniquement via JOIN FETCH ou @EntityGraph dans les queries spécifiques.



readOnly = true systématiquement

Désactive le dirty checking Hibernate, libère des ressources et permet au driver JDBC d'optimiser les requêtes.



Projections pour les lectures

Les opérations en lecture seule doivent utiliser des DTO ou projections. Charger une entité complète uniquement si on a l'intention de la modifier.



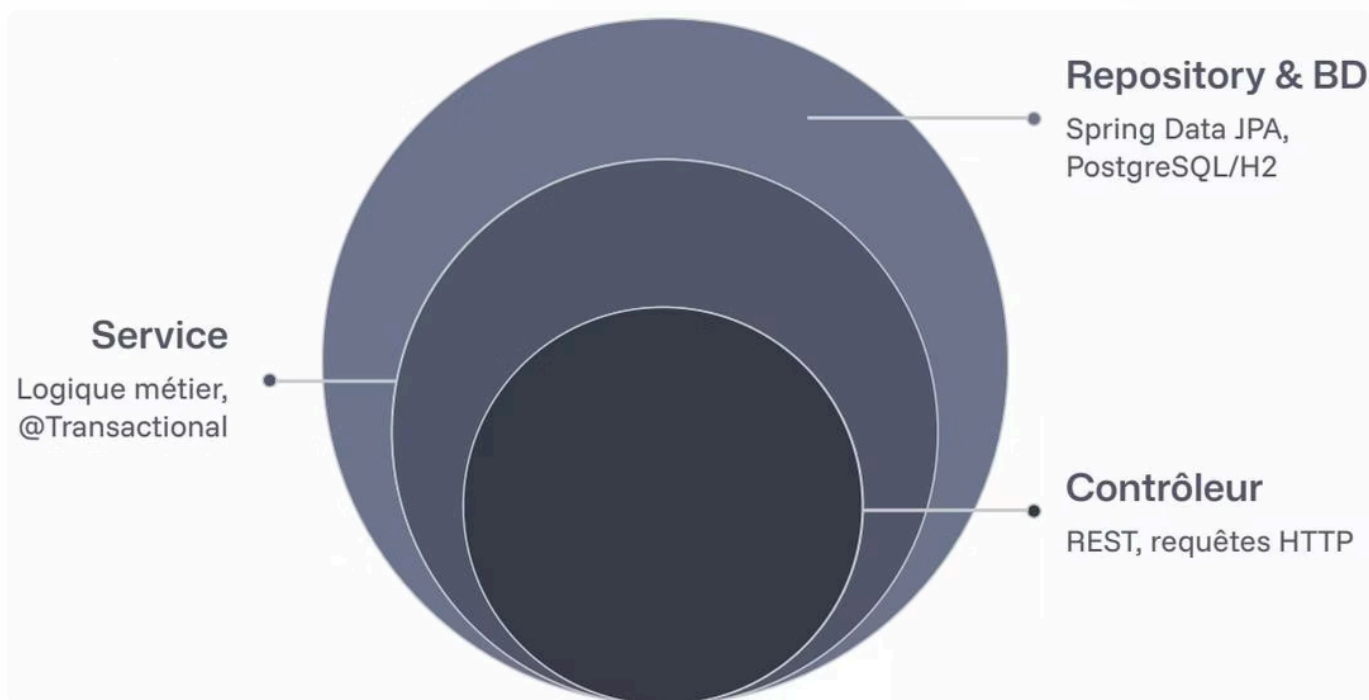
Mesurer avant d'optimiser

Toujours profiler avec des données réalistes. L'optimisation prématurée est source de complexité inutile.

Stratégie de chargement – guide de décision

Contexte	Stratégie	Pourquoi
Lecture d'une entité et ses associations	JOIN FETCH ou @EntityGraph	Évite le N+1, une seule requête SQL
Liste paginée, peu d'associations	Projections / DTO	Charge uniquement les données utiles
Entité à modifier (CRUD)	findById + LAZY	Charge l'entité complète pour la mise à jour
Export / rapport volumétrique	Projections + pagination ou Stream	Évite les OutOfMemoryError
Données de référence peu modifiées	Cache L2 ou @Cacheable	Élimine les requêtes répétées
Mise à jour en masse (> 1000 entités)	@Modifying bulk update	Exponentiellement plus rapide que save()

Architecture en couches Spring Data JPA



Controller

Gère les requêtes HTTP, délègue au Service. Utilise des DTO — jamais d'entités JPA exposées directement.

Service

Contient la logique métier. Porte les annotations @Transactional.

Repository

Étend JpaRepository, gère les accès aux données via Hibernate.

Spring Data JPA en chiffres

80%

Réduction du code

Par rapport à une approche JDBC classique grâce aux repositories auto-générés.

10x

Gain de performance

Observé en remplaçant des requêtes N+1 par des JOIN FETCH sur des tables de 100K lignes.

95%

Cas couverts

Des besoins de requêtage couverts par les query methods et JPQL, sans SQL natif.

5min

Mise en place

Pour configurer une couche de persistance complète avec Spring Boot, JPA et H2.

Comparatif des approches de persistance Java

Critère	JDBC pur	JPA / Hibernate	Spring Data JPA	jOOQ / MyBatis
Productivité	Faible	Bonne	Excellente	Bonne
Contrôle SQL	Total	Moyen	Moyen	Élevé
Courbe d'apprentissage	Faible	Élevée	Modérée	Modérée
Performance tuning	Manuel	Complexe	Complexe	Direct
Portabilité SGBD	Faible	Élevée	Élevée	Faible
Adapté entreprise	Non	Oui	Oui ✓	Oui (SQL lourd)

Anti-patterns à éviter absolument

Entités exposées en API

Exposer les entités JPA via REST couple l'API au modèle de données et peut provoquer des boucles infinies de sérialisation. Toujours utiliser des DTO.

findAll() sans pagination

Sur des tables volumineuses, peut provoquer des OutOfMemoryError en production. Toujours imposer une limite ou une pagination.

@Transactional sur les controllers

Allonge inutilement les transactions, monopolise des connexions du pool. Les transactions appartiennent à la couche service.

ddl-auto=create en production

Peut détruire des données à chaque redémarrage. Utiliser `validate` et gérer les migrations avec Flyway ou Liquibase.

Checklist de revue de code JPA

Entités et mapping

- Toutes les associations sont-elles LAZY ?
- equals/hashCode sur l'id métier ?
- Flyway gère le schéma (pas ddl-auto=create) ?

Requêtes et repositories

- Les listes ont-elles une pagination ?
- Les lectures utilisent-elles des projections ?
- Le problème N+1 a-t-il été analysé ?

Transactions et performance

- Les services de lecture ont-ils readOnly = true ?
- Les transactions sont-elles au niveau Service ?
- Les données de référence sont-elles cachées ?